

Modelos de Deep Learning

Definición de Attention

Universidad ORT Uruguay

17 de Noviembre, 2025

Definición de Attention

Nos basamos fuertemente en el paper [Vaswani et al., 2017].

*An attention function can be described as **mapping** a **query** and a set of **key-value pairs** to an **output**, where the query, keys, values, and output are all **vectors**.*

*The output is computed as a **weighted sum** of the values, where the weight assigned to each value is computed by a **compatibility function** of the query with the corresponding key.*

Es decir, la primera oración de la cita dice que si tenemos

- Espacio de queries: $\mathcal{Q}$
- Espacio de keys: $\mathcal{K}$
- Espacio de values: $\mathcal{V}$
- Espacio de diccionarios (key-value pairs): $\{\mathcal{K} : \mathcal{V}\}$

un mecanismo de **attention** es una función

$$\text{Attention} : \mathcal{Q} \times \{\mathcal{K} : \mathcal{V}\} \rightarrow \mathcal{V}$$

Supondremos que $\mathcal{Q} = \mathcal{K}$.

La segunda oración dice que si denotamos:

■ $\mathcal{Q} = \mathbb{R}^p$ y $\mathbf{q} = [q_1, \dots, q_p]$,

■ $\mathcal{K} = \mathbb{R}^p$ y $\mathbf{k} = [k_1, \dots, k_p]$,

■ $\mathcal{V} = \mathbb{R}^d$ y $\mathbf{v} = [v_1, \dots, v_d]$,

entonces la función Attention es de la forma:

$$\text{Attention}(\mathbf{q}, \{\mathbf{k}_i : \mathbf{v}_i\}_{i=1}^n) = \sum_{i=1}^n \alpha_i(\mathbf{q}, \{\mathbf{k}_i\}_{i=1}^n) \mathbf{v}_i$$

Las funciones α_i calculan la **compatibilidad** entre $\mathbf{q}$ y las keys.

Casos extremos de compatibilidad

Example (Extremo: winner-takes-all)

$$\alpha_i(\mathbf{q}, \{\mathbf{k}_i\}_{i=1}^n) = \mathbb{1}_{\{i=\arg \max_j (\mathbf{q} \cdot \mathbf{k}_j)\}}.$$

Example (Extremo: indecisión)

$$\alpha_i(\mathbf{q}, \{\mathbf{k}_i\}_{i=1}^n) = \frac{1}{n}$$

Caso intermedio: softmax

Compatibilidad mediante softmax de productos escalares:

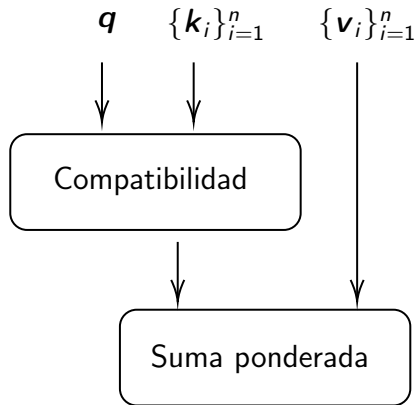
$$\alpha_i(\mathbf{q}, \{\mathbf{k}_i\}_{i=1}^n) = \text{Softmax}_i[\mathbf{q} \cdot \mathbf{k}_1, \dots, \mathbf{q} \cdot \mathbf{k}_n].$$

Los extremos se obtienen ajustando una temperatura τ :

■ $\tau = 0$: **winner-takes-all**;

■ $\tau = \infty$: **indecisión**.

Diagrama general de Attention



Attention para múltiples queries

Si tenemos varias queries $\{\mathbf{q}_j\}_{j=1}^m$, la función devuelve los m outputs:

$$\text{Attention}(\{\mathbf{q}_j\}_{j=1}^m, \{\mathbf{k}_i : \mathbf{v}_i\}_{i=1}^n) = \left\{ \sum_{i=1}^n \alpha_i(\mathbf{q}_j, \{\mathbf{k}_i\}_{i=1}^n) \mathbf{v}_i \right\}_{j=1}^m$$

Notación matricial

Esto se puede escribir en notación matricial de manera compacta:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}(\mathbf{Q}\mathbf{K}^T) \mathbf{V}$$

Entradas:

- **Q**: matriz de queries,
- **K**: matriz de keys,
- **V**: matriz de values.

Shapes de las matrices

Los shapes son:

$$\mathbf{Q} \sim (m, p),$$

$$\mathbf{K} \sim (n, p),$$

$$\mathbf{V} \sim (n, d).$$

Aquí:

■ m : número de queries,

■ n : número de keys.

Producto matricial $\mathbf{QK}^\top$

El producto matricial produce una matriz $m \times n$:

$$(\mathbf{QK}^\top)_{j,i} = \mathbf{q}_j \cdot \mathbf{k}_i.$$

Es decir:

$$\mathbf{QK}^\top = \begin{bmatrix} \mathbf{q}_1 \cdot \mathbf{k}_1 & \cdots & \mathbf{q}_1 \cdot \mathbf{k}_n \\ \vdots & \ddots & \vdots \\ \mathbf{q}_m \cdot \mathbf{k}_1 & \cdots & \mathbf{q}_m \cdot \mathbf{k}_n \end{bmatrix}$$

Cada fila corresponde a una query.

Softmax fila a fila

Aplicamos softmax **fila por fila**:

$$\text{Softmax}(\mathbf{QK}^\top) = \begin{bmatrix} \text{Softmax}(\mathbf{q}_1 \cdot \mathbf{k}_1, \dots, \mathbf{q}_1 \cdot \mathbf{k}_n) \\ \vdots \\ \text{Softmax}(\mathbf{q}_m \cdot \mathbf{k}_1, \dots, \mathbf{q}_m \cdot \mathbf{k}_n) \end{bmatrix}$$

Cada fila es una distribución sobre los n values.

Multiplicación por $\mathbf{V}$

Finalmente multiplicamos por $\mathbf{V}$:

$$\text{Softmax}(\mathbf{QK}^\top) \mathbf{V} \in \mathbb{R}^{m \times d}.$$

Fila j :

$$\sum_{r=1}^n \text{Softmax}_r(\mathbf{q}_j \cdot \mathbf{k}_1, \dots, \mathbf{q}_j \cdot \mathbf{k}_n) \mathbf{v}_r$$

Es decir: **una combinación ponderada de las filas de $\mathbf{V}$.**

Self-attention

[Ambartsoumian and Popowich, 2018]: racconto sobre Attention y su historia.

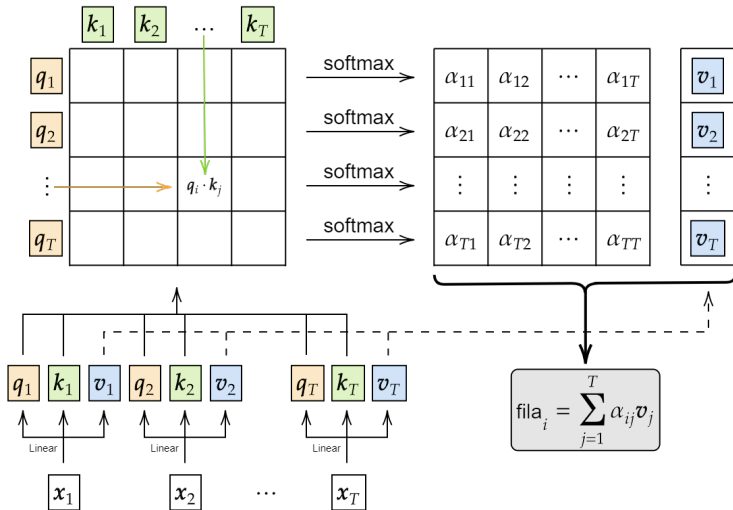
*Originally [Bahdanau et al., 2014] described attention as the process of computing a **context vector** for the next decoder step that contains the most relevant information from all of the encoder hidden states by performing a weighted average on the encoder hidden states. How much each encoder state contributes to the weighted average is determined by an **alignment score between that encoder state and previous hidden state of the decoder.***

*More generally, we can consider the previous **decoder state as the query vector**, and the **encoder hidden states as key and value vectors**. The output is a weighted average of the value vectors, where the weights are determined by the compatibility function between the query and the keys. [...]*

Definición de Self-Attention

Self-attention is the process of applying the attention mechanism outlined above to every position of the **source sequence**. This is done by creating three vectors (**query**, **key**, **value**) for each sequence position, and then applying the attention mechanism for each position $\mathbf{x}_i$, using the $\mathbf{x}_i$ query vector and key and value vectors for all other positions. As a result, an input sequence $\mathbf{X} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$ of words is transformed into a sequence $\mathbf{Y} = (\mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_n)$ where $\mathbf{y}_i$ incorporates the information of $\mathbf{x}_i$ as well as **how $\mathbf{x}_i$ relates to all other positions in $\mathbf{X}$** . The (query, key, value) vectors can be created by applying **learned linear projections** or using **feed-forward layers**.

Self-Attention: diagrama



Resumen

- Dada una query $\mathbf{q}$,
- values $(\mathbf{v}_1, \dots, \mathbf{v}_n)$,
- y keys $(\mathbf{k}_1, \dots, \mathbf{k}_n)$,

calculamos el output $\mathbf{z}$:

$$\mathbf{z} = \sum_{j=1}^n \alpha_j \mathbf{v}_j$$
$$\alpha_j = \frac{\exp f(\mathbf{k}_j, \mathbf{q})}{\sum_{i=1}^n \exp f(\mathbf{k}_i, \mathbf{q})}$$

Esto es, α_j es la softmax de las compatibilidades $f(\mathbf{k}_i, \mathbf{q})$ entre $\mathbf{k}_i$ y $\mathbf{q}$.

Scaled dot-product attention

Se suele usar la **scaled dot-product function** como función de compatibilidad:

$$f(\mathbf{k}, \mathbf{q}) = \frac{\mathbf{k}\mathbf{q}^\top}{\sqrt{d_k}}$$

donde d_k es la dimensión de las keys.

Cita de [Ambartsoumian and Popowich, 2018]:

*This scaling is done to improve numerical stability as the **dimension** of keys, values, and queries **grows**.*

Self-Attention en forma matricial

Este cálculo se puede hacer para la source entera en paralelo agrupando las queries, keys, y values en matrices $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ como hicimos anteriormente:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax} \left(\frac{\mathbf{Q}\mathbf{K}^{\top}}{\sqrt{d_k}} \right) \mathbf{V}$$

Invarianza bajo permutaciones

*The attention mechanism is **completely invariant to sequence ordering**, thus self-attention networks need to incorporate **positional information**.*

*Three main techniques have been proposed to solve this problem: **adding sinusoidal positional encodings** or **learned positional encoding to input embeddings**, or **using relative positional representations in the self-attention mechanism**.*

Bibliografía



Ambartsoumian, A. and Popowich, F. (2018).

Self-attention: A better building block for sentiment analysis neural network classifiers.

In *Proceedings of the 9th Workshop on Computational Approaches to Subjectivity, Sentiment and Social Media Analysis*, pages 130–139. Association for Computational Linguistics.



Bahdanau, D., Cho, K., and Bengio, Y. (2014).

Neural machine translation by jointly learning to align and translate.

arXiv preprint arXiv:1409.0473.



Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017).

Attention is all you need.

In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc.